IN2033: Peer review of Requirements and High-Level Design Document

Review conducted by Team: 4
Review conducted on the document submitted by Team: 5
Review completed on: March 8, 2026
Review submitted for assessment on: March 09, 2026

Review goals:

- **G1: Completeness**
  - G1.1: Completeness of Requirements
    - This is a comprehensive document that goes over a good amount of use cases with the majority of the use cases being detailed and complete.
  - G1.2: Completeness of High-Level Design
    - This is a comprehensive document that has detailed java implementations and complete high-level design diagrams.
  - G1.3: Completeness of Testing Plan
    - This is a comprehensive document that has a good amount of test cases testing subsystem functionality, but, there are a few test cases missing when testing validation.
- **G2: Correctness**
  - G2.1 Correctness of Requirements
    - This is a comprehensive document with detailed use cases, but, some use cases have observed logical errors in their execution.
  - G2.2 Correctness of High-Level Design
    - This is a comprehensive document that has correct java implementations and cohesive high-level design diagrams.
  - G2.3 Correctness of Testing Plan
    - This is a comprehensive document that has a good amount of test cases testing subsystem functionality, but, there are a few data type errors in some of the expected outcomes.
- **G3: Quality of the document**
  - See Deliverable Quality Assessment

Review Log

Document:	IPOS-CA High Level Design and Requirements Document
Version:	Version 1
# of Issues:	12
Review Date:	March 08, 2026

Attendees	Read document (Y/N)	Time spent preparing
Herman Sildnes	Y	2 Hr
Yamin Ahmed	Y	1 Hr
Joanna Ayeni	Y	1 Hr
Kanzah Hussain	Y	1 Hr
Jenis Khorshed	Y	1 Hr
Leon Seferi	Y	1 Hr
Vladyslav Semanyshyn	Y	1 Hr

Issue number:	**Section/page:**	**Identified by:**	**Issue:**	**Fix:**
1	Section 5.2.1.2 / p.08 - Use Case ID: 3	Yamin	Logic error, merchant cannot self activate an account. SA must approve first	Update flow to include a "pending approval" state managed by SA subsystem
2	Section 7.6/ p.45 - IPUPaymentAPI	Yamin	Data type error, expected outcomes are described as visual changes only, ignoring the underlying data return types (json/int/boolean)	Update "expected outcome" to include the specific data type or API response code (e.g. 200 OK)
3	Section 7.6/ p.45 - IPUCommAPI and IPUPaymentAPI	Kanzah	submitPayment() method Test IDs 2,3,4,5,6 have omitted parameters and replaced them with "...". sendEmail also has omitted parameters with … except for the first Test ID	Insert valid parameters for all tests as done so for the first TestID of each method.
4	Section 7.6	Kanzah	Many tests have the Expected Outcome as "Exception Illegal Argument" without explaining what happens/doesn't happen as a result of this. The expected outcome	Explain the Expected Outcome instead of just writing the error message. For example, in ICACatalogAPI on page 44

			should clearly explain the outcome of the error.	sendOrderDetails(...): boolean, the Expected Outcome could be “Product not shipped”.
5	Section 7.6/ p.43-45 - ISAOrderAPI and ICACatalogAPI	Leon	Inconsistencies with the return type. In the interface seen on page 39, ISAOrderAPI -> getCatalogue() returns ‘String[]’. This indicates it being returned as a list or array. The tests then, however, later describe it as returning ‘string’, a single text value. Same issue also found with viewPreviousOrders()	Update test case expected outcomes to match interface exactly. For example, expect a non-empty array/list of strings for catalogue/orders rather than a single string.
6	Section 7.6/ p.44 - viewInvoices	Leon	Test cases don't align with interface definition. Interface defines it as returning ‘String[]’ but test cases labels it as void and expects a boolean return type. Correctness issue.	Update test case to align with interface or vice versa.
7	Section 7.6/ p.45 - IPUCommsAPI	Leon	Inconsistency with method names seen in test case and interface. Interface defines getOrderUpdate(int orderID, String status), but test plan only refers to getOrderStatus() and makes no reference to the original interface method.	Rename the test cases to use the same method name, depending on which is the intended. One name should be used consistently for one method at a time.
8	Section 7.6/ p.46 - IPUPaymentAPI	Leon	Input coverage incomplete for submitPayment(). They test zero payment, missing name, negative card number and expired card but not invalid cardType. Given cardType is an explicit interface parameter, specific card validation	Add a test for an unsupported cardType (like an unknown provider). State the expected outcome - such as false if unsupported.

			would be appropriate here.	
9	Section 5.2.1.2/ p.11 - Use case 7	Vladyslav	The use case has a precondition that says “The products delivered match what was ordered”, but doesn’t state what happens if they don't match. It makes the use case slightly incomplete	Alternative flow can be added for problems with delivery
10	Section 5.2.1.2/ p.23 - Use case 24	Vladyslav	“The manager is logged in” is listed as a precondition, but the primary actor is time. Automatic monthly process shouldn’t depend on manager being logged in	Remove the manager login precondition
11.	Section 5.2.1.2, p.12-13	Jenis	The IPOS document specifies that all reports must be manually generated and downloaded as spreadsheets, with no requirement for a live, auto-refreshing dashboard that displays key metrics at-a-glance when users log in.	Add a new use case (SA-RPT-07) for a real-time analytics dashboard that automatically displays visual KPI cards, trending charts, and configurable alerts without requiring manual report generation.
12	Section 7.5, P.21-24	Jenis	The Java interfaces (ISALoginAPI, ISAMemberAPI, ISAOrderAPI, IPUCommsAPI) define method parameters but contain no specifications for how validation errors—such as invalid product IDs, empty passwords, or malformed emails—should be visually displayed to users in the interface.	Document validation rules for each interface parameter and specify UI feedback mechanisms including inline error messages, field highlighting, error banners, and user-friendly error text for every validation scenario.

Deliverable Quality Assessment:

Criterion:	Rank:	Justification:
Appropriate title, page numbering, version control	Excellent	Title, page numbering and version control all as expected
Introduction, and Purpose and Scope	Good	The introduction is comprehensive, but the purpose and scope could make it more clear that the document covers the CA subsystem
Use of language appropriate to audience, and Spelling & Grammar	Excellent	Use of technical language appropriate to the expected audience. Nothing to note on spelling and grammar
Clear layout and structure	Good	The overall structure of the document is good, but would have benefitted from improved formatting in certain places, like avoiding tables that could fit on one page from overflowing onto another page.
Appropriate use of graphics and diagrams	Excellent	Diagrams are legible, captioned and correct.